An Introduction to Linear Quadratic Control by Modelling a Mass-Spring-Damper System

Abstract*—This paper introduces the linear quadratic regulator (LQR) methodology for optimal state-feedback control, applied to a mass-spring-damper system. The theoretical foundations of LQR are presented, followed by a Python-based simulation using standard scientific computing libraries. The spring-mass-damper system is modelled in its state-space representation, and simulation results are analysed for both the uncontrolled and controlled cases.*

I. INTRODUCTION

Linear quadratic regulation (LQR) is an optimal state-feedback control technique for dynamical systems described in state-space form. In general, any system governed by an nth-order differential equation can be represented as a system of n coupled first-order ordinary differential equations[1]. The state-space representation reduces the system to:

$$\dot{x}(t) = Ax(t) + Bu(t) \quad (1)$$

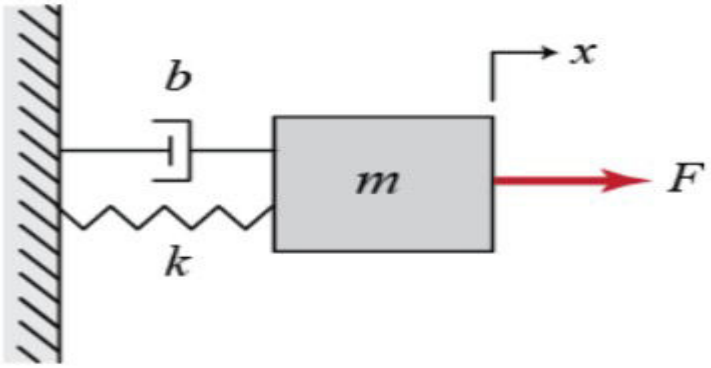
$$y(t) = Cx(t) + Du(t) \quad (2)$$

where (1) describes the state dynamics and (2) describes the system output. Classical controllers such as the proportional-integral-derivative (PID) controller are widely used but are less effective for complex, multi-input multi-output (MIMO) systems. The LQR framework is better suited to such systems, as it simultaneously accounts for all state variables and optimises a quadratic cost criterion.

This paper introduces the LQR design methodology and demonstrates its application to a spring-mass-damper system through numerical simulation.

II. SYSTEM MODEL

The spring-mass-damper system consists of a mass m attached to a wall via a spring of stiffness k and a viscous damper of coefficient c, subject to an external control force u.



A. Mass-Spring-Damper Dynamics

The system is governed by the second-order differential equation:

$$m\ddot{x} + c\dot{x} + kx = u \quad (3)$$

where m is the mass (kg), c is the damping coefficient (N·s/m), k is the spring constant (N/m), u is the control input force (N), and x is the displacement (m).

B. State-Space Representation

Defining the state variables $x_1 = x$ (position) and $x_2 = \dot{x}$ (velocity), the state vector is $x = [x_1, x_2]^T$. Dividing (3) by m and substituting yields:

$$\dot{x}_1 = x_2 \quad (4)$$

$$\dot{x}_2 = -(k/m)x_1 - (c/m)x_2 + u/m \quad (5)$$

In matrix form, $\dot{x} = Ax + Bu$, where:

$$A = [[0, 1], [-k/m, -c/m]], \quad B = [[0], [1/m]] \quad (6)$$

Since position x_1 is the measured output, $y = x_1$ and $C = [1\ \ 0]$, $D = 0$.

III. LINEAR QUADRATIC REGULATOR

A. State-Feedback Control

Under full state feedback, the control input takes the form:

$$u(t) = -Kx(t) \quad (7)$$

where K is the feedback gain matrix. Substituting (7) into (1) gives the closed-loop dynamics:

$$\dot{x}(t) = (A - BK)x(t) \quad (8)$$

Closed-loop stability is determined by the eigenvalues of $(A - BK)$. The design problem is to select K such that these eigenvalues lie in the left half of the complex plane with the desired transient characteristics.

B. Cost Function

The LQR framework selects K by minimising the following quadratic cost function:

$$J = \int_0^\infty (x^T Q x + u^T R u)\, dt \quad (9)$$

where $Q \geq 0$ is the state weighting matrix, penalising deviation of the states from equilibrium, and $R > 0$ is the control weighting matrix, penalising actuator effort. Positive semi-definiteness of Q and positive definiteness of R ensure convexity and the existence of a unique optimal solution.

C. Interpretation of Weighting Matrices

The diagonal entries of Q assign a penalty to each state variable. A large entry q_{ii} causes the optimiser to reduce the corresponding state error more aggressively. R penalises control magnitude; a large R yields a conservative controller that limits actuator effort at the cost of slower convergence. Since no universally optimal Q and R exist, their selection reflects the designer's trade-off between performance and control cost.

D. Optimal Control Law and Riccati Equation

Minimising J subject to (1) yields the optimal gain[2]:

$$K = R^{-1}B^T P \quad (10)$$

where P is the unique symmetric positive-definite solution to the Continuous Algebraic Riccati Equation (CARE):

$$A^T P + PA - PBR^{-1}B^T P + Q = 0 \quad (11)$$

This closed-form solution is computationally efficient and exact for linear time-invariant systems.

IV. SIMULATION SETUP

A. Numerical Implementation

The simulation was implemented in Python 3.11.13 using the scipy and numpy[3] library for easy computation and the matplotlib library for plotting the responses of the systems The CARE (11) was solved via solve_continous_are()[4], which gives as the matrix P,from which the optimal feedback K can be calculated using(10). State trajectories were obtained by integrating the closed-loop ODE using scipy.integrate.solve_ivp[4]

with the RK45 method over a 20 s horizon in which the transients died out and the system reached its steady state. The initial condition was $x(0) = [1, 0]^T$, corresponding to a unit displacement with zero initial velocity.

B. Controller Parameters

System and controller parameters are listed in Table I. $Q = \text{diag}(10, 1)$ places a tenfold greater penalty on position error than on velocity error, reflecting a position-regulation objective. $R = 1$ provides a balanced trade-off between convergence speed and actuator effort. The resulting gain is $K \approx [1.74, 1.68]$ with closed-loop poles at $-1.09 \pm 1.60j$.

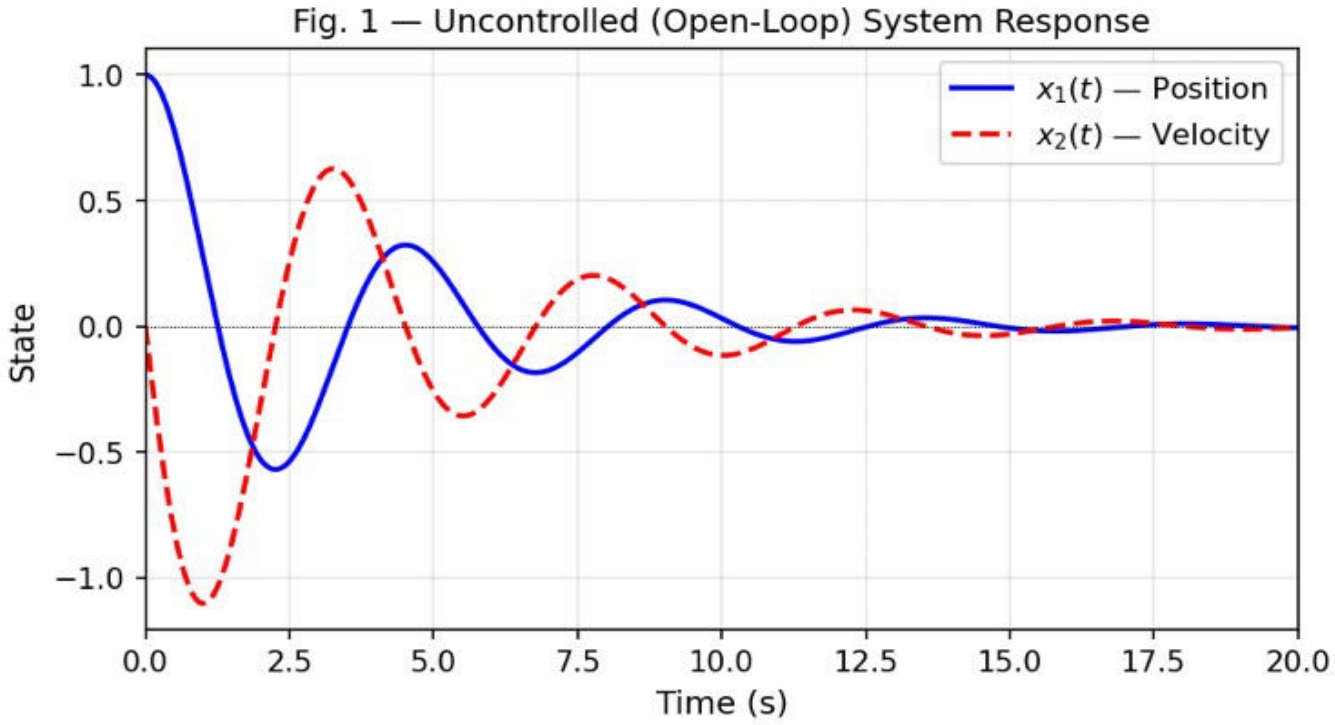
TABLE I — System and Controller Parameters

Parameter	Symbol	Value	Unit	Notes
Mass	m	1.0	kg	Normalized
Damping coefficient	c	0.5	N·s/m	Light damping
Spring constant	k	2.0	N/m	Stable open-loop
State weight	Q	diag(10, 1)	—	Penalizes x_1
Control weight	R	1.0	—	Balanced effort
Initial conditions	x(0)	$[1, 0]^T$	m, m/s	Unit displacement

V. RESULTS

A. Uncontrolled System Response

With $u = 0$, the system evolves as $\dot{x} = Ax$ from $x(0) = [1, 0]^T$. The open-loop eigenvalues are $\lambda = -0.25 \pm 1.40j$, giving a damping ratio $\zeta \approx 0.177$ and natural frequency $\omega_n \approx 1.41$ rad/s. The position $x_1(t)$ exhibits slowly decaying sinusoidal oscillations with period ≈ 4.5 s, as shown in Fig. 1. While the system is nominally stable, the slow energy dissipation motivates active feedback control.

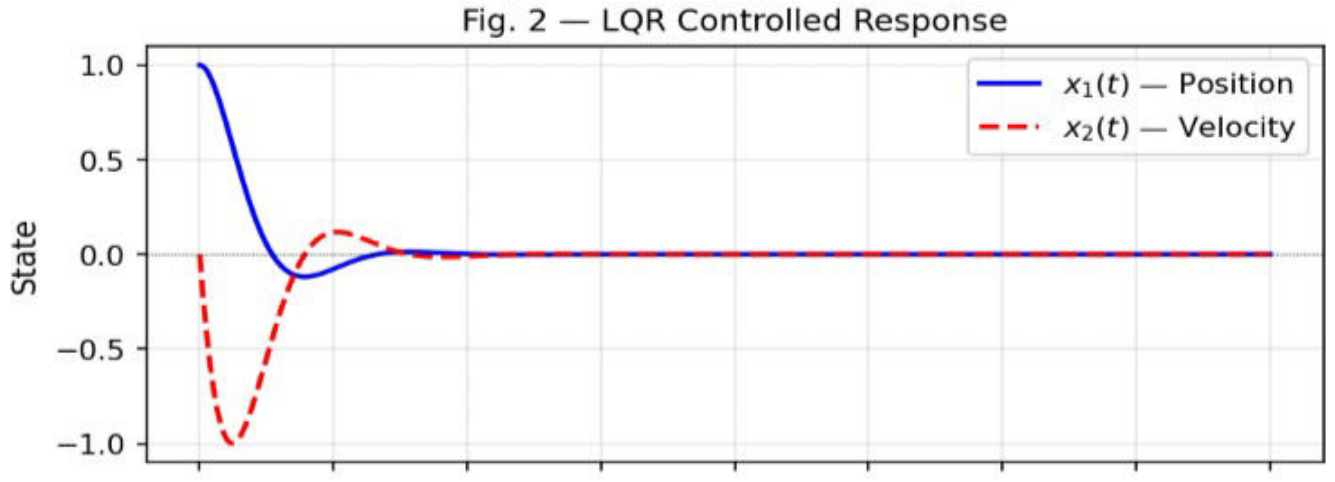


[Fig. 1 — Open-loop state trajectories]

Fig. 1. Open-loop state trajectories $x_1(t)$ and $x_2(t)$ from $x(0) = [1, 0]^T$.

B. *LQR Controlled Response*

Under LQR control with gain K ≈ [1.74, 1.68], the closed-loop poles are placed at −1.09 ± 1.60j. As shown in Fig. 2, both states converge to the origin. The position settles within approximately 1.32 s (2% criterion) with an overshoot of 11.8%, a marked improvement over the open-loop settling time of more than 15 s.



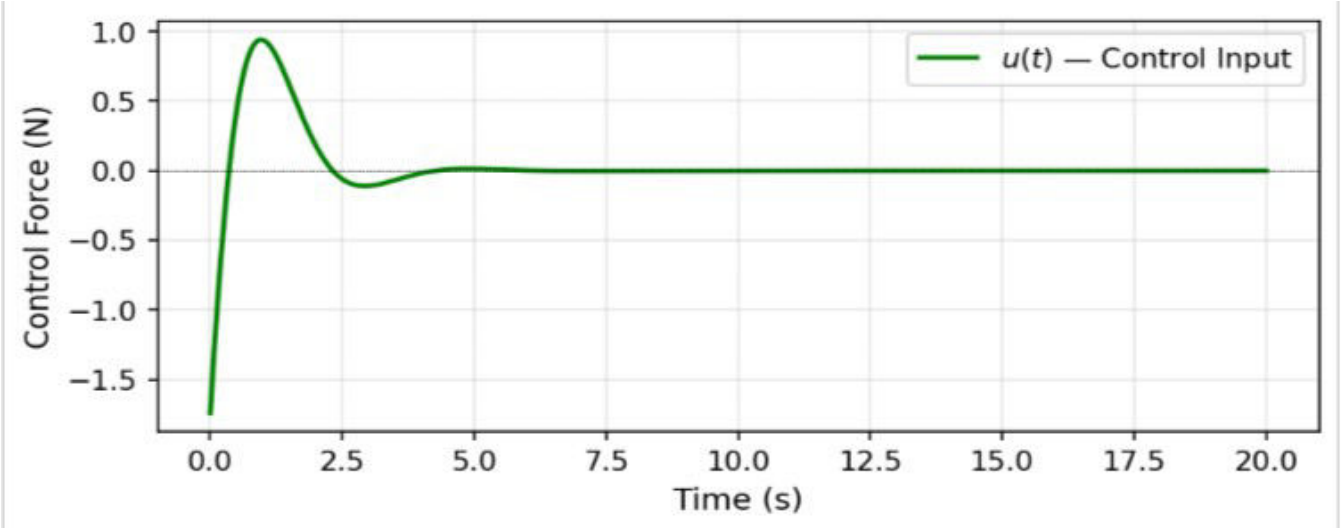
[Fig. 2 — LQR controlled state trajectories]

Fig. 2. LQR-controlled state trajectories $x_1(t)$ and $x_2(t)$ from $x(0) = [1, 0]^T$.

C. *Control Input*

Fig. 3 shows the control input u(t) = −Kx(t) over time. The input peaks at approximately 1.74 N at t = 0 and

decays to zero as the states converge, confirming the energy-optimal nature of the LQR controller.



[Fig. 3 — Control input u(t)]

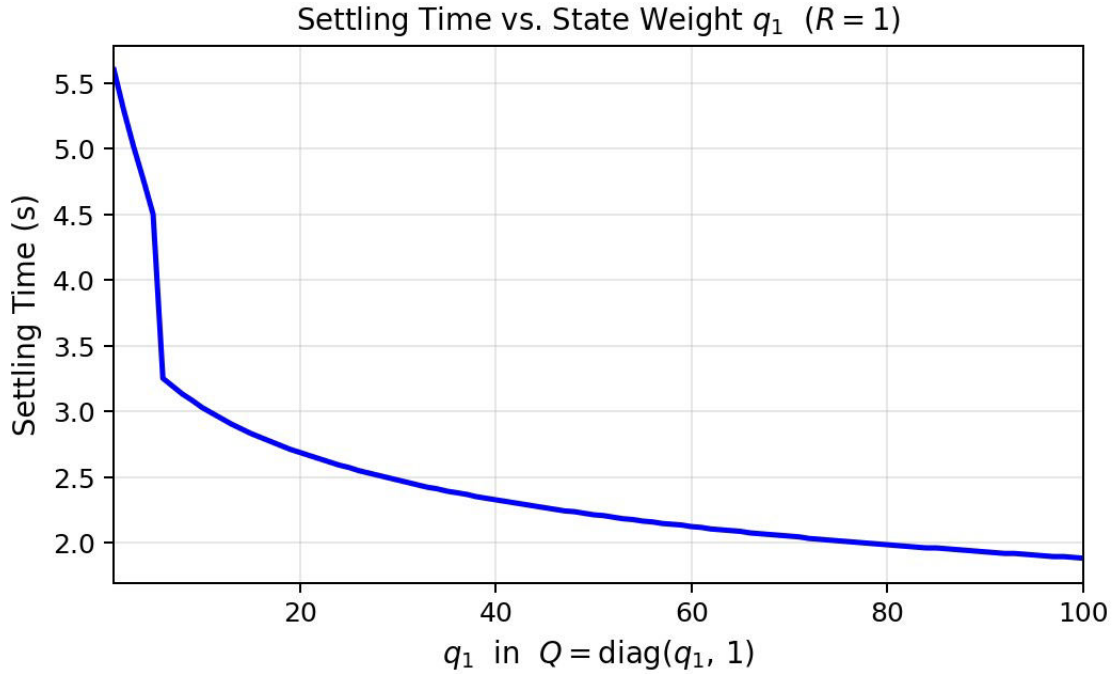
Fig. 3. LQR control input u(t) over time.

D. *Effects due to variation of the weight matrices Q and R*

Since the value of the gain matrix that we get after solving the CARE[4] depends on the value of the weight matrices Q and R ,the design of the controller is inherently non-universal in nature,ie,the controller design depends heavily upon the problem specification and the cost we allot to the required state vs the actuator cost.

In our spring-mass-damper system , since we want the mass to reach its equilibrium position when released from the initial state X=[1,0] (initial position=1 and velocity=0) which implies that for Q=W.I (I is an identity matrix),greater the Value of W(weights),smaller will be the settling time ‘t’,ie they follow an inverse relation and we get faster convergence and reduced settling time.This is verified:-

VI.

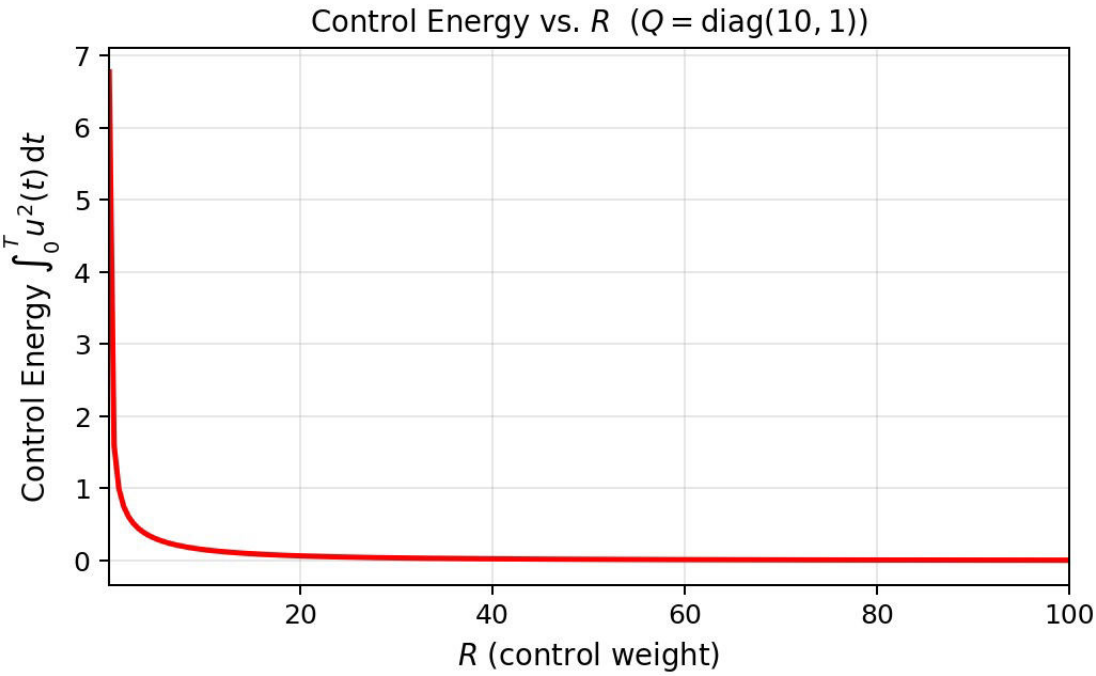


We

measure the actuator effort be measuring the energy of the control force signal u(t) by using the definition of the energy of a signal(In practice we use the discretized version, since we take discretized time steps):-

$$E = \int_0^T u^2(t)\,dt$$

The weights of the R matrix specify the amount of energy expended in reaching the corresponding desired state. We write the R matrix as a column matrix and each element of the matrix signifies the weight of the energy to be used,Larger the value of R,lesser is the energy used,ie an inverse relation



These results point to a profound point that the design of an LQR controller is highly dependent on the use case and the nature of the system. In space systems,the energy spent matters a lot therefore we may allot higher weights to the R matrix than Q but in defence critical systems like missiles and UAVS higher preference may be given to the Q matrix since reaching the desired state faster and minimizing the position error is of utmost importance.

CONCLUSION

The spring-mass-damper system was controlled by applying the input u=-Kx,ie,the gain matrix that we solved for using the continuous algebraic Riccati equation. It should be noted that for different values of the Q and R matrix provided Q,R>0, we well get different values of P which will lead to different values of K hence a different controlling force. The nature of the controlling force depends upon the user,ie,whether they give more weight to reaching the desired state quickly or reducing the actuator effort(in our case the control force u(t)).

The primary objective of controller design is to achieve stable and desirable system behavior, and in this

sense, optimal control methods such as Linear Quadratic Regulator (LQR) and classical approaches such as PID control share the same fundamental goal of regulating system dynamics. In LQR design, the Algebraic Riccati Equation provides an analytical solution for linear systems with quadratic cost functions; however, it is not the only possible approach to solving optimal control problems. Modern control research also explores learning-based and computational optimization techniques, including gradient descent methods, reinforcement learning frameworks, and convex optimization formulations. These approaches provide additional flexibility for handling complex or high-dimensional systems. Overall, LQR provides a mathematically systematic and computationally efficient method for state feedback control of linear dynamical systems. herefore it depends upon the designer of the controller to set the values of Q and R according to the need of the problem resulting in a rich design experience.

References

[1] [Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control

by Steven L Brunton and Nathan J Kutz .]

[2] [“What Is Linear Quadratic Regulator (LQR) Optimal Control” by Brian Douglus,MATLAB Tech Talk]

[2] [Feedback systems:An introduction for scientists and engineers by Richard Murray and Astrom .]

[3] [Harris, C.R., Millman, K.J., van der Walt, S.J. et al. Array programming with NumPy. Nature 585, 357–362 (2020).]

[4] [The Riccati Equation and its importance for LQR control" by Brian Douglus,MATLAB Tech Talk]